DT2213 Project

Artificial Life, Sonified

Members: Sze Wing Chan, Oskar Lövgren, Barry John Wootton, Yu Lu

Supervisor: Maurizio Berta

Introduction

This project explores the generation of music that embodies both novel, unpredictable elements and discernible, coherent structures or patterns. The central idea is to explore how the complex dynamics of self-organized, bio-inspired artificial life systems can serve as a source for real-time artificial music improvisation. This is an important area of study as it offers a computational approach for musical communication that thrives on a balance between surprise and familiarity, as well as requiring real-time interaction and generation, such as those found in improvisation or in video games. In an era where music consumption is becoming more participatory and interactive, systems that allow for emergent, user-influenced musical structures open up new possibilities for engagement and expression, both artistically and experientially.

By drawing inspiration from natural systems where intricate global patterns emerge from simple local interactions (Blackwell & Young, 2004), this project seeks to explore alternate pathways for musical communication and expression, and potentially for interactive musical experiences. The primary objective is to leverage the entropy within these swarming algorithms to create musical novelty, while simultaneously harnessing their capacity for emergent structures to create patterning which appeals to the musical expectation and coherence of the listener.

Background

Self-organizing principles in natural systems, where collective behavior and emergent structures arise from local interactions rather than centralized control, draws parallels to improvised music, where interactions at lower levels, such as the sound event level, eventually develop into the meso- or macro-structure of a piece (Blackwell & Young, 2004). The exploration of such systems in music is driven by their potential to generate outputs that are "neither random nor unpleasant" (Blackwell & Young, 2004, p. 124), offering a unique blend of complexity, novelty and order. This interest is part of a broader history in algorithmic composition, which has seen various approaches including the use of artificial life and multi-agent models to generate musical material (Duarte, 2020). Historically, the desire to compose with processes that balance predictability and surprise also resonates with mid-20th-century explorations of information theory in music, which framed aesthetic content in terms of fluctuations between total randomness and total redundancy (Patteson, 2024). Algorithmic composers then began to see creation as a process of imposing constraints on an initially random field of possibilities, or a "total matrix of possibilities" (Patteson, 2024, p. 9).

However, this domain is not without its challenges. A primary concern in algorithmic music generation is the "too random" problem, where the output may lack the coherence or patterning that listeners often associate with musicality (Patteson, 2024). There's a subjective line where an excess of information, or randomness, can exceed cognitive processing abilities, moving from engaging originality to incomprehensibility. Conversely, the complete absence of authorial

imposition, while potentially liberating for the listener in line with ideas from chance music, also risks a lack of engaging musical direction if not carefully managed. Another critical issue lies in the mapping process, which is translating the behaviors and data from the algorithmic system into audible musical results (Doornbusch, 2002). The challenge is to create mappings that are neither so direct and predictable as to be trivial, nor so complex and opaque that the connection between the algorithm's processes and the musical output is lost. This is particularly pertinent given our aim to avoid very direct mappings, as advised by our supervisor. Instead, we aim for a balance where a listener might perceive a correspondence between the system's activity and its sonic effects without it being arbitrary or overly simplistic.

Early algorithmic composers like Xenakis and Dodge demonstrated diverse mapping strategies, from direct to more abstract relationships (Doornbusch, 2002). Furthermore, algorithmic music generation offers a powerful means to address practical issues like excessive repetition, a concern particularly relevant in contexts like video game music where dynamic, reactive and infinitely variable scores are desirable (Duarte, 2020). As illustrated by Hunt et al. (2002), mappings between physical control and musical output can vary from simple one-to-one relationships to highly cross-coupled, many-to-many configurations. While the latter are often less transparent, they tend to afford greater expressive depth and a more instrument-like responsiveness. This project seeks to navigate these complexities by focusing on how the emergent behaviors and inherent stochasticity of self-organizing particle-life systems could be thoughtfully mapped to musical features, aiming for that compelling balance of "novelty with structure".

Self-organizing systems more broadly fall under the umbrella of artificial life (ALife), where simplified simulations of living systems, spanning across cellular automata, evolutionary algorithms, swarming and flocking models, and particle life models, all of which explore how complex structure can emerge from local interactions. Among these, particle life models use simple attraction and repulsion rules between different particles to give rise to emergent clustering and group behavior, offering a well-suited foundation for this project's aims.

One of the classic algorithms that have been used in previous ALife music systems is Conway’s Game of Life, a two-dimensional cellular automaton (CA) where the cells’ life are determined by a set of rules and the state of its neighboring cells (Izhikevich et al., 2015). Miranda (2002) presents CAMUS, an early music system based on the Game of Life. CAMUS maps the evolution of the cellular patterns to pitch and timing, using a 2D grid as a musical representation. Although the mappings are relatively straightforward, they demonstrate the potential of CA systems to generate structured, repeatable musical events from stochastic dynamics. Another system that uses that Game of Life as foundation is Ogawa and Kuhara’s *Life Game Orchestra* (2009). Although this system has a less innovative musical mapping than CAMUS (i.e. detects predefined cell patterns and maps them to tones on a musical scale), it introduces an interactive performance element, where users generate cells through body movement captured by a camera, which is an example of incorporating user agency without breaking the evolving nature of the system. Although these systems can produce emergent and

novel music, it is needed to consider that they often lack the cultural references or musical conventions listeners expect, making the outputs feel alien or overly abstract (Miranda, 2002).

More recent work in ALife based music systems have led to the development of libraries that facilitate real-time interaction between self-organizing ALife simulations and musical systems. One notable example is *Tölvera*, a Python library designed for musical composition, interaction, and performance with “basal agencies” (i.e. low-level, non-symbolic entities), including particle-life models (Armitage et al., 2024). *Tölvera* offers built-in Open Sound Control (OSC) support for music interaction, enabling artists to sonify or musically respond to emergent behaviors without implementing the underlying artificial life mechanics themselves. The system emphasizes modularity and artistic exploration, and has been used in a series of creative projects to test its capabilities.

A key challenge in the use of non-adaptive AI systems for music generation lies in the need for creators to effectively translate algorithmic behaviors into musically meaningful outcomes (Kaliakatsos-Papakostas et al., 2020), and for users to comprehend and engage with those translations. In the context of this project, where the goal is to develop a real-time, autonomous composition system based on the emergent dynamics of a self-organizing artificial life model, this translation process takes on added importance. The system is also intended to serve as a creative tool for non-expert users, allowing them to influence musical outcomes without requiring traditional compositional knowledge. To support this aim, emphasis is placed on creating intuitive mappings and simplifying the path to musically pleasant results, rather than prioritizing complex or highly detailed control mechanisms. Previous work targeting similar goals (Unehara & Onisawa, 2005) has shown that constraining compositional elements—such as limiting melodies to a single key and restricting phrase lengths to fixed bar counts—can effectively enable non-experts to engage in music-making. Notably, studies have found that user involvement in the generative process increases perceived enjoyment of the resulting musical output (Unehara & Onisawa, 2005).

Method

System Selection

We looked at various potential artificial-life/self organising systems such as swarming systems, cellular automata, L-systems etc. based primarily on their potential affordances in terms of musical manipulation, interactivity and temporal aspects such as continuity. Some considerations were:

- Using real video input (pre-recorded) of schools of fish or flocks of birds, using object detection.
- Using L-Systems to generate a finite complex structure which would result in a finite piece of music.
- Using some form of evolutionary simulation such as cellular automata.

A secondary criteria was existing and readily available system implementations of the above that are easy to work with, seeing that the musical mapping and output was the focus and not the simulation system itself, and due to the time constraints of the project. We selected the particle-life system because it would allow us to manipulate the system (unlike the pre-recorded video) as well as potentially interact with it in real-time.

Tölvera[1], the ALife music system mentioned in Background, initially appeared to be a fitting tool, as given its focus on simulating particle life and its support for OSC, it could simplify the development. However, as we began implementation, we encountered multiple technical issues due to the documentation being relatively fragmented and insufficient, resulting in observability problems of system state. Hardware wise, there were compatibility issues that arose as dependency libraries assumed Apple Silicon hardware, yet the project members were using different OS platforms. As a result, we pivoted to using Clusters[2], a web-based particle life simulator that provides real-time visual feedback developed by Jeffrey Ventrella. Clusters' source code is publicly available on GitHub[3] and it is built with vanilla JavaScript and HTML, making it easy for us to read and modify the system and implement MIDI-based integrations with WebMIDI. While it lacks some of Tölvera's extensibility, its simplicity and responsiveness allowed us to focus more directly on the musical experimentation, mapping implementation, and interaction designs.

[1] https://iil.is/research/tolvera
[2] https://www.ventrella.com/Clusters/
[3] https://github.com/Ventrella/Clusters

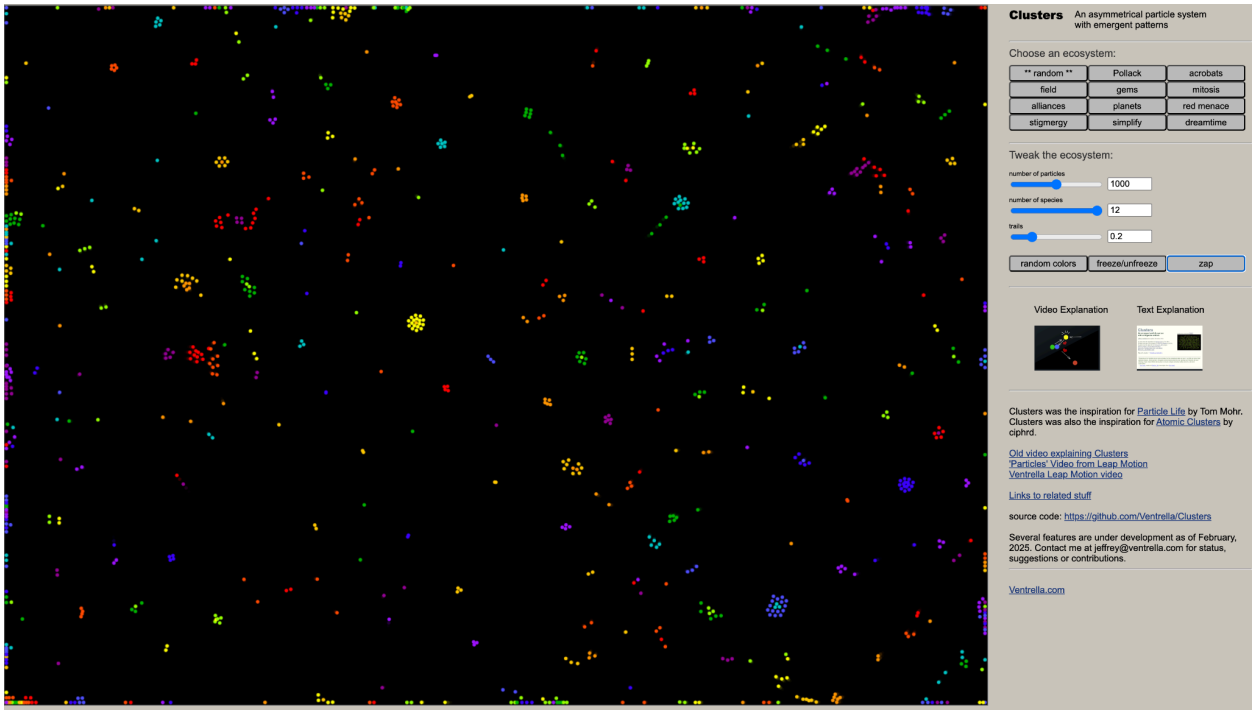


Figure 1 - The original Clusters system

Figure 1 shows the original Clusters system with original UI, before any adjustments made by us. The Clusters system has multiple “Ecosystems”, which we found the *Alliances* ecosystem to be the best for our use case because over time, clear alliances or groups start to emerge which we could potentially use as different interacting agents.

The modified system can be found in our forked repository[4]. All subsequent code references below can be found in this repository. All code additions made by the group for this project can be seen in the commit history of the fork, most of which are also noted in the descriptions below.

Monitoring of the Particle System

Before deciding how we would monitor the particle system, we set some guiding criteria for the eventual sound output. We wanted the output to be musical and not totally random or dissonant, this means that we needed to have some level of control or transparency, so we needed to stay clear of arbitrary mappings. This would also serve the potential for human interaction with the system. Therefore, we wanted there to be somewhat of a correspondence between the visuals and the audio output, even though this is not entirely necessary as the system could just be a source of entropy and structure – a means to an end – the musical output.

[4] https://github.com/wincsw/MusicProject

Extracting features

We decided on “sampling” or “listening” to a specific area of the system, as this allows one to listen to specific areas where an interesting pattern may be forming. We therefore had to determine when a group of particles enter or leave the sample area, and therefore also had to detect the groups. Groups can form with multiple species (colours) and therefore we needed to determine the dominant species within a group. We decided that each species can have the ability of a certain musical transformation, thereby a group of a certain species being an agent of a certain musical change. The properties of each group in relation to the sampling area determines the aspects of that musical change. The properties used were the sample area entry position, entry speed and group size. The entry position is one of 12 positions, corresponding to the 12 outer positions in the Circle of Fifths, starting at the 3 O’clock position, where 1=D, 2=G and so on.

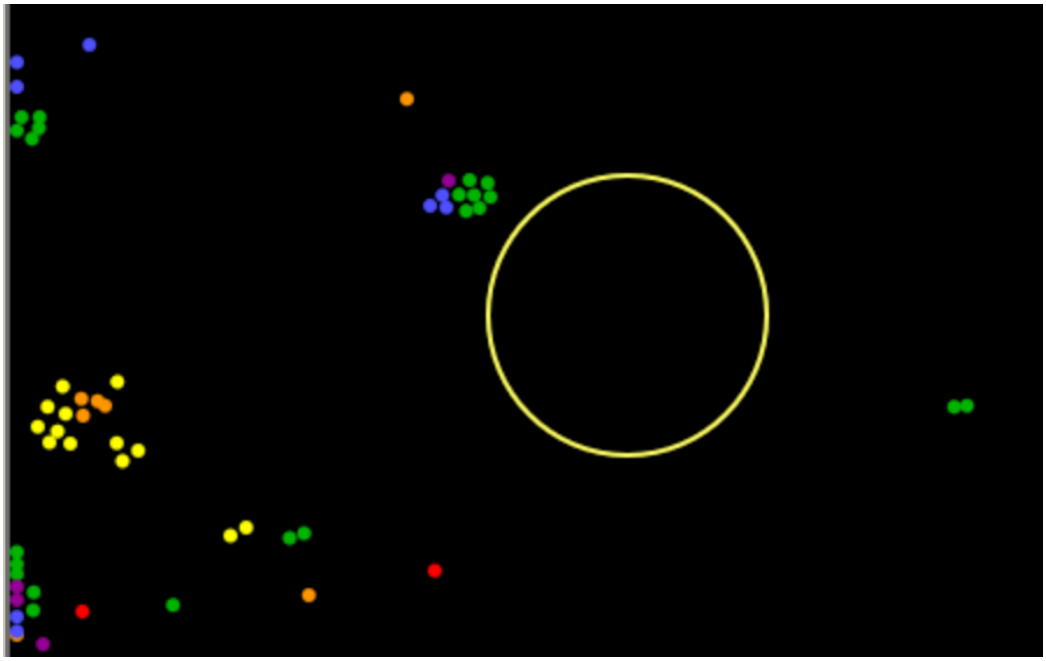


Figure 2 - Adjustable Sampling Area

Setting the sample area

The sample area can be placed or moved by positioning the cursor and pressing the ‘s’ key. The area is shown as illustrated in Figure 2.

Determining entry and grouping

Determining entry to the sample area was simple enough using circle center and euclidean distance. The notion of what constitutes a group is more complex because in reality it is not only determined by proximity but also by similar velocity. We therefore approximate the detection and formation of a group as explained below.

The procedure below can be found in *Clusters.js*, line 434-494.

- For each particle, determine whether it falls within the sample area
- If so, extend a perpendicular line backwards from the point of entry with length of group radius. *Helpers.js* function *extendFromEntry.*
- Check if any of the other particles are within the group radius (e.g: behind the leading particle that has entered the sample area). See the green circle in Figure 3.
- If so, and the total number of particles within the group radius is greater than the configurable group size, then calculate group properties and add a group object to a groups array.

Particles can only belong to one group, we therefore also need to:

- When forming a group, check that the particle is not part of another group.
- When a group leaves the sampling area, the group and its member particles need to be disbanded so they are free to form future groups.
- To simplify a group leaving, a group is deemed to have left when its leader particle (the first of the group to enter the area) has left the sample area.
- A group leaving is signified by a red circle.

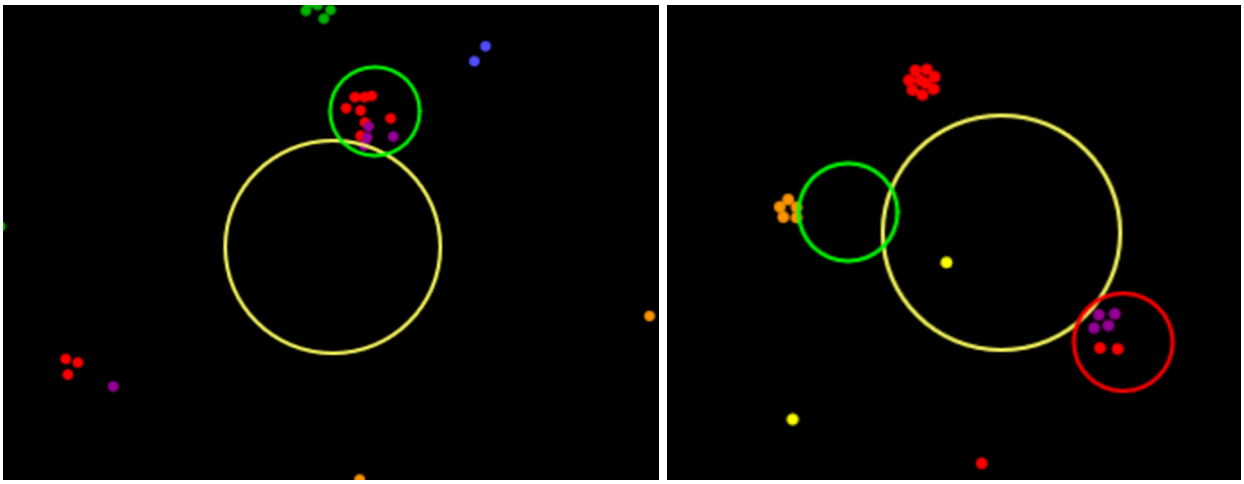


Figure 3 - Entry (green) and Exit (red)

Note: The green circle is visualised when any particle enters the sample area, even if there are insufficient particles to form the group, it's only the search area for a potential group to be formed. The red circle only visualises when the leader of the group exits the sample area, even if the members have separated from the group and already left the area.

Drawing of the sample and group circles is done in *Clusters.js*, lines 685-727.

Determining group properties

Entry position: We calculate the angle from the center of the sample circle to entry point, calculate the angle and calculate which of the 12 slices it falls into (2*PI / 12). *Helpers.js* function *getSlice*.
Group speed: Magnitude of the velocity of the leader particle (existing system already has a *getMagnitude* method in the *Vector2D.js* class).
Group species: Tally the species of each particle member and return the species with the highest count. *Helpers.js* function *mostFrequentSpeciesInSubset*.
Group size: length of the groupset array.

Creating the group object: *Helpers.js* function *createGroup.*

These properties are subsequently normalised/scaled to MIDI values and mapped to musical outputs.

UI Panel

The original UI panel shown in Figure 1 was replaced, the original GUI takes up a relatively large part of the webpage, leaving less space for the visualisation of the particle simulation. We use the *lil-gui*[5] package as a replacement with a floating control panel that can be hidden. In this new floating control panel, we added UI elements for controlling the size of the sample area while removing some irrelevant UI elements, such as tweaks for the number of species (fixed at 6 for our project), trails, and colors.

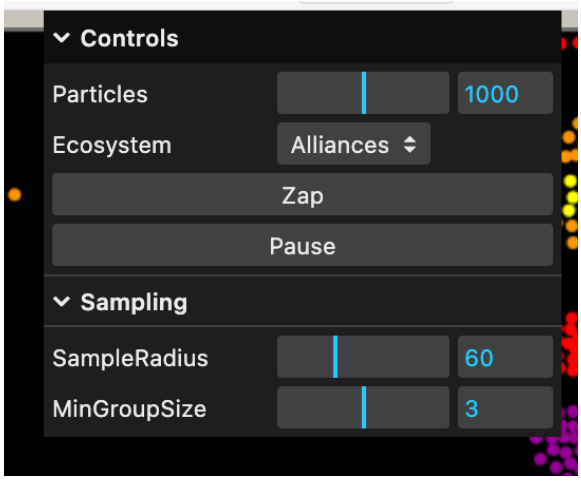


Figure 4 - Floating and foldable UI panel

[5] https://github.com/georgealways/lil-gui

User Interaction

Limited user interaction with the particle system can be performed by clicking and dragging the particles; this interaction was present in the original Clusters system. This simple interaction can be interesting however, especially when used with the *Pause* function, as one can pause the particle system and rearrange things or even manually drag a desired group into the sample area. This combined with the transparent mapping, especially that of the red species (pitch), can be used like an instrument.

Sound communication protocol

Events occurring from groups entering the sample area along with the abovementioned group properties are mapped to MIDI messages (CC and Note) using a WebMIDI library[6]. This library is integrated directly into our fork of the Clusters application, in *MIDI.js*.

MIDI messages are routed via virtual MIDI buses, using the Mac IAC Driver[7], to an Ableton Live 12 session setup to receive messages from the specified buses.

Mapping to music outputs

Initial mapping strategies for sonifying particle system data were based on the assumption that each particle species would independently control all aspects of a sound event: note triggering (on/off), pitch, velocity, and continuous modulation via MIDI CC messages. However, as individual species only provides a few classes of information that could make for meaningful and intuitive mapping to audio production and manipulation (group size, group size, entry position), this approach proved to be impractical.

As a result, a more distributed mapping approach was adopted in which different species were assigned specialized roles within the creation of the sonification. With this approach, one species might govern the harmonic and pitch-related aspects of the music, while another controls dynamic parameters such as volume of different sonication elements (instruments). A third species could be dedicated to modulation tasks, influencing sound-shaping elements like filter cutoff, resonance, or effect parameters through CC data.

It was decided for the sonification, that it would consist of a few different complimenting sounds, to allow for a variety of mappings. In order to avoid overly repetitive or rhythmically rigid textures, sound synthesis was prioritized over sample-based playback. Virtual instruments were chosen to allow for real-time modulation and dynamic variation. Additionally, percussive and drum-based elements were deliberately excluded to maintain a degree of rhythmic ambiguity and to emphasize continuous, evolving textures.

The sonic framework was composed of four primary musical elements:

[6] https://webmidijs.org/
[7] https://help.ableton.com/hc/en-us/articles/209774225-Setting-up-a-virtual-MIDI-bus

- A synth pad providing harmonic content through chords or two-note harmonies,
- A synth pluck sound,
- A bright synth stab,
- A synth bass for low-frequency support.

Mappings were conceived for up to six particle species, each assigned to influence one or more of these musical layers according to their designated behavioral characteristics and roles within the system.

Species	**Mapping 1**	**Mapping 2**
Red	Entry position → Pitch	Group size → chord partials
Orange	Group speed → Pluck modulation rate	Group size → Pluck modulation depth
Yellow	Group speed → Stab modulation rate	Group size → Stab modulation depth
Green	Group size → Sequence Stab	N/A
Blue	Group size → Sequence Bass	N/A
Purple	Entry quadrant → Select Pluck/Stab/Bass	Group speed → Volume of Pluck/Stab/Bass

Table 1 - Initial mapping idea

These mappings are implemented in *MIDI.js*, lines 99-230.

Live session

A project session was set up in Ableton Live Suite 12. Before connection to the Clusters was considered, four MIDI channels were set up with different virtual instruments, and using stand-in MIDI data, the sonification sound and mix was tested and tuned to ensure a listenable result when the actual midi data was used.

From discrete to continuous

Since only one species in the system was responsible for controlling pitch, specific methods were needed to determine how each instrument channel would be triggered and kept active. For the pad channel, which is directly linked to the pitch-controlling species, MIDI note messages were used to trigger playback. The instrument's built-in Hold function allowed the pad to sustain a note until a new one was received, enabling continuous sound based on the species' input.

For the pluck, stab, and bass channels, continuous playback was not possible due to the short, non-sustaining nature of the sounds used. Instead, these channels were controlled by Ableton Live's SQ sequencer. Each sequencer was set to its longest loop length and programmed to

receive the same pitch as the pad. This created repeating 16-step note patterns. Although this approach was slightly less direct, it still allowed for particle behaviors—such as group size or species type—to influence how often notes were played through sequence selection.

To allow for smooth, ongoing changes to sound parameters—such as filter or effect settings—even though the system receives only occasional events from the simulator, an LFO (Low Frequency Oscillator) was used as an intermediate tool. Rather than using particle values to set a fixed parameter value, those values controlled the LFO's modulation depth. This means the LFO kept moving the parameter up and down over time, but the size of that movement depended on the particle input. This helped the system respond to particle behaviors in a way that felt fluid and continuous.

Custom MIDI effects

To reinterpret or reshape the incoming event-triggered MIDI data to fit its use case, each MIDI channel in the template Live project is equipped with a custom *MIDI Effect Rack*. The leveraging of Ableton Live's MIDI-effects are essential to the setup chosen where the simulator sends a limited set of note information. The MIDI Effect Racks enable transformations such as octave shifting, chord expansion, and modulation triggering.

The following MIDI effects were used within Ableton Live to process the incoming MIDI data from the Clusters simulator:

- Chord Select:
  A five-chain MIDI Effect Rack using Ableton's Chord MIDI effect. Each chain applies a different harmonic structure, ranging from simple two-note intervals to richer six-note chords. Chain selection is controlled by a macro mapped across the full 0–127 MIDI CC range, enabling dynamic harmonic variation.

- Sequence Select:
  A four-chain MIDI Effect Rack with pre-programmed MIDI sequences of increasing rhythmic complexity. The Seq Select macro control determines which sequence is active, with the 0–127 CC range divided evenly among the chains.

- Impact Modulation:
  A modulation setup using Ableton's Envelope MIDI and LFO devices. MIDI note input triggers an ADSR envelope which modulates the rate of an LFO. The resulting signal is mapped to parameters such as filter cutoff or modulation depth, creating a decaying modulation effect that enhances impact and expressivity. To further the sense of disturbance and change on group entry, each LFO also slightly controls the panning of its sound, creating a sense of movement and has the potential to redistribute the sounds spatially between impacts.

Results

The final system presents ten mappings between the behaviors of the Clusters particle species and the musical sonification. Each species within the particle system is assigned one or two mappings to parameters shaping the sonification, and may also send a MIDI note to a specific instrument channel.

Species	Mapping 1	Mapping 2	Send note to instrument
Red	Entry position → Set root pitch: 24 - 36,	Group size → Number of notes in pad chord: 2 - 6	On entry: Pad
Orange	Group speed → Pluck Impact modulation start-rate: 0 - 40 Hz	Group size → Pluck LFO modulation depth: 0 - 100%	On entry: Pluck
Yellow	Group speed → Stab Impact modulation start-rate: 0 - 40 Hz	Group size → Stab LFO modulation depth: 0 - 100%	On entry: Stab
Green	Group size → Select sequence Stab: 1 - 4 (2 - 5 active steps)	N/A	N/A
Blue	Group size → Select sequence Bass: 1 - 4 (2 - 5 active steps)	N/A	N/A
Purple	Entry quadrant → Select Pluck/Stab/Bass	Group speed → Set gain of selected element: -20.0 - 0.0dB	N/A

Table 2 - Final mapping structure

Chord Selection

The chord selector (Red Mapping 2) creates harmony with the root pitch by adding upshifted notes as input CC value increases, beginning from a minor seventh and going all the way up to an octave and perfect fifth.

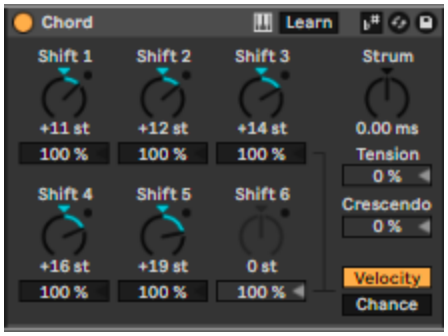


Figure 5 - Ableton Live Chord module playing six notes including the input root pitch

Impact modulation mapping

The pluck and stab sounds are modulated using LFOs connected to the Clusters system (Species Orange and Yellow). The LFO modulation is controlled by the group size and speed, as the group size increases, the modulation depth increases with it, in the end modulating in the full range specified in the modulated parameter. The group speed specifies the starting rate of the LFO modulation. As a group enters the sample area, the ADSR envelope increases the start rate value by 50% and then over a long decay returns back to the start modulation rate, to create the impact effect. Both LFOs are mapped to modulate the panning of their sound between 50L and 50R when modulation depth is 100%.



Figure 6 - Ableton Live Envelope MIDI

The pluck LFO is mapped to a high pass filter cutoff and the release of the synths amplitude envelope. The high pass filter range is inverted in order for the bandwidth of the sound to increase as the species group size increases, same as the release time.

The stab LFO is mapped to the built in filter envelope amount in the synth, moving between fully closed to fully open as the the LFO depth is 100%.

The pluck and stab instrument channels are modulated using LFOs linked to input from Species Orange and Yellow, respectively. Each LFO's behavior is influenced by two parameters: group size and group speed. As the group size increases, the modulation depth of the LFO increases

proportionally, reaching full modulation when the group size reaches its maximum. The group speed sets the LFO's initial rate of modulation.

When a group enters the sample area, an ADSR envelope is triggered, temporarily increasing the LFO rate by 50%. The rate then decays back to its original value over time. Both LFOs are mapped to stereo panning, modulating the sound between 50L and 50R at maximum depth.

- Pluck Channel (Species Orange):
  The LFO modulates both the cutoff frequency of a high-pass filter and the release time of the synth's amplitude envelope. The filter mapping is inverted, so as group size increases, the cutoff frequency lowers, expanding the audible bandwidth. The same logic applies to the release time, which increases with group size.

- Stab Channel (Species Yellow):
  The LFO is mapped to the filter envelope amount within the synthesizer. At maximum modulation depth, the filter envelope moves from fully closed to fully open, shaping the brightness and presence of the stab sound based on particle activity.

MIDI Channel and CC number distribution

For intelligibility, the CC controls for each channel in Live is assigned to its own MIDI channel and the same type of control are assigned the same CC numbers on their respective MIDI channels (See Table 3). In order not to interfere with reserved CC numbers of instruments used in the template project CC numbers 120-125 are used. This does not abide by the MIDI standard[8] but can be easily changed in the *MIDI.js* file.

MIDI Ch. / CC Number	120	121	122	123 - Occupied by Live	124	125
1	Chord Select					
2		LFO Rate	LFO Depth			Gain
3		LFO Rate	LFO Depth			
4					Seq Select	Gain
5					Seq Select	Gain

[8] https://studiocode.dev/resources/midi-cc/

Table 3: MIDI Channel and CC number distribution. Rows MIDI Channels, Columns MIDI CC Numbers, Colors representing Live instrument channel, Red: Pad, Orange: Pluck, Yellow: Stab, Blue: Bass.

References

Armitage, J., Shepardson, V., & Magnusson, T. (2024). Tölvera: Composing With Basal Agencies. *Proceedings of the International Conference on New Interfaces for Musical Expression*. https://iil.is/pdf/2024_nime_armitage_et_al_tolvera.pdf

Blackwell, T., & Young, M. W. (2004). Self-organised music. Organised Sound, 9(2), 123–136. https://doi.org/10.1017/s1355771804000214

Doornbusch, P. (2002). Composers' views on mapping in algorithmic composition. Organised Sound, 7(2), 145–156. https://doi.org/10.1017/S1355771802002066

Duarte, A. E. L. (2020). Algorithmic interactive music generation in videogames. SoundEffects - An Interdisciplinary Journal of Sound and Sound Experience, 9(1), 38–59. https://doi.org/10.7146/se.v9i1.118245

Hunt, A., Wanderley, M. M., & Paradis, M. (2002). The importance of parameter mapping in electronic instrument design. *Journal of New Music Research*, *32*(4), 429–440. https://doi.org/10.1076/jnmr.32.4.429.18853

Izhikevich, E., Conway, J., & Seth, A. (2015). Game of life. *Scholarpedia*, *10*(6), 1816. https://doi.org/10.4249/scholarpedia.1816

Kaliakatsos-Papakostas, M., Floros, A., & Vrahatis, M. N. (2020). Chapter 13 - Artificial intelligence methods for music generation: a review and future perspectives. In X.-S. Yang (Ed.), Nature-Inspired Computation and Swarm Intelligence (pp. 217–245). https://doi.org/10.1016/B978-0-12-819714-1.00024-5

Miranda, E. R. (2002). Sounds of artificial life. *Proceedings of the 4th Conference on Creativity & Cognition*, 9–16. https://doi.org/10.1145/581710.581736

Ogawa, K., & Kuhara, Y. (2009). Life Game Orchestra as an interactive music composition system translating cellular patterns of automata into musical scales. *Proceedings of the International Conference on New Interfaces for Musical Expression (NIME)*. https://doi.org/10.5281/zenodo.1177647

Patteson, T. (2024). Entropy, melody, beauty: composing with information theory. Culture, Theory and Critique, 1–11. https://doi-org.focus.lib.kth.se/10.1080/14735784.2024.2343700

Unehara, Muneyuki & Onisawa, Takehisa. (2005). Music Composition by Interaction between Human and Computer.. New Generation Comput.. 23. 181-191. 10.1007/BF03037494. http://dx.doi.org/10.1007/BF03037494